



Lab 05 – Introduction to Verilog HDL

Name:	ID:	Section:
-------	-----	----------

Objectives

After performing this lab, student should be able to:

- Understand syntax of Verilog HDL
- Write modules for digital logic in Verilog
- Create Testbench for functional verification

Equipment/Apparatus

None

This lab consists of four different sections. The description of each section is given below:

Section	
a) <u>Background</u>  The aim of this section is to introduce the basic syntax of Verilog HDL.	20
b) <u>Hands-on Verilog HDL</u>  In this part, we will do some practical examples in Verilog HDL. We will use software tool named Xilinx Vivado to write HDL codes.	80
c) <u>Functional Verification</u>  In this part, we will learn to simulate and verify the functionality of a Verilog modules developed, using HDL, in the previous part. Vivado will be used for functional verification.	50
d) <u>Exercise</u>   Small exercise to practice either at home or in the lab.	30



a. Background

Verilog is a hardware description language and is intended for describing and modeling a digital system at various levels of design. From modest beginning in 1984, the Verilog has become an industry standard as a result of extensive use in the design of digital systems. Following section briefly describes the syntax of Verilog HDL and covers some basic lexical elements (i.e. language constructs, keywords and formats) to write a Verilog HDL code.

i. Lexical Elements

The basic lexical conventions used by Verilog HDL are similar to those in the C programming language. Verilog HDL is a case-sensitive language. All keywords are in lowercase.

Here we will only discuss some of the lexical conventions that are new to Verilog. For a detailed reference, Verilog Language Reference manual or any other reference on Verilog may be consulted.

Identifier – An *identifier* gives a unique name to an object, such as `counter`, `seven_segment`, `el2`, etc. It is composed of letters, digits, the underscore character (`_`), and the dollar sign (`$`). `$` is usually used with a system task or function.



The first character of an identifier must be a letter or underscore.



Verilog is a *case-sensitive language*. Thus, `data-bus`, `Data-bus`, and `DATA_BUS` refer to three different objects. To avoid confusion, we should refrain from using the case to create different identifiers.

White space – White space, which includes the space, tab, and newline characters, is used to separate identifiers and can be used freely in the Verilog code. We can use proper white spaces to format the code and make it more readable.

Comments – A *comment* is just for documentation purposes and will be ignored by a compiler. Verilog has two forms of comments. A one-line comment starts with `//`, as:

```
// This is a comment
```

A multiple-line comment is encapsulated between `/*` and `*/`, as shown below:

```
/* This is comment line 1.  
   This is comment line 2.  
   This is comment line 3. */
```

ii. Number representation

There are two types of number specification in Verilog: sized and unsized.



Sized numbers

Sized numbers are represented as

```
[sign][size]'[base format] [number]
```

[sign] is written in the case of signed numbers. [size] is written only in decimal and specifies the number of bits in the number. Legal base formats are decimal ('d' or 'D'), hexadecimal ('h' or 'H'), binary ('b' or 'B') and octal ('o' or 'O'). The number is specified as consecutive digits from 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f. Only a subset of these digits is legal for a particular base. Upper-case letters are legal for number specification. Here are some examples

```
4'b1111    // This is a 4-bit binary number
12'h2F7    // This is a 12-bit hexadecimal number
16'd255    // This is a 16-bit decimal number
```

Unsigned numbers

Numbers that are specified without a [base format] specification are decimal numbers by default. Numbers that are written without a <size> specification have a default number of bits that is simulator- and machine specific (must be at least 32).

```
23456      // This is a 32-bit decimal number by default
'hc3       // This is a 32-bit hexadecimal number
'o21       // This is a 32-bit octal number
```

Negative numbers

Putting a minus sign before the size for a constant number can specify negative numbers. Size constants are always positive. It is illegal to have a minus sign between <base format> and <number>.

```
-8'd3      // 8-bit negative number stored as 2's complement of 3
4'd-2      // illegal specification
```

iii. Data Types

Four basic values are used in most data types:

- 0: for "logic 0, or a false condition
- 1: for "logic 1", or a true condition
- z: High impedance, floating state
- x: for an unknown value

Verilog has two main groups of data types: net and register.

Nets

Nets represent connections between hardware elements. Just as in real circuits, nets are like circuit traces/wires which do not store value. Nets have values continuously driven onto them by the outputs of devices that are connected to.



Nets are declared primarily with the keyword **wire**. Nets are one-bit values by default unless they are declared explicitly as vectors.

```
wire p;                // one 1-bit signal
wire p0, p1;           // two 1-bit signals
wire [7:0] data1;      // 8-bit data
wire [31:0] addr;      // 32-bit address
wire [0:7] revers-data; // ascending index should be avoided
```



While the index range can be either descending (as in [7:0]) or ascending (as in [0:7]), the former is preferred since the leftmost position (i.e., 7) corresponds to the MSB of a binary number.



It is possible to address bits or parts of a vector. For example, `data1[3]` refers to the bit 3 of a wire `data1` declared above. `addr[2:0]` three least significant bits of a vector `addr`

The term **wire** and net are often used interchangeably. **The default value of a net is z.** Nets get the output value of their drivers. If a net has no driver, it gets the value **z**.

Registers

Registers represent storage data elements. Registers retain value until another value is placed onto them. Do not confuse the term registers in Verilog with hardware registers built from edge triggered flip-flops in real circuits. In Verilog, the term register merely means a variable that can hold a value. Unlike a net, a register does not need a driver. Verilog registers do not need a clock as hardware registers do. Values of registers can be changed anytime in a simulation by assigning a new value to the register.

Register data types are commonly declared by the key word `reg`. **The default value for a reg data type is x.** An example of how registers are declared and used is shown below:

```
reg reset;           // declares a single bit a variable that can hold
                     // its value
```

iv. Modules

Verilog provides the concept of a module. A module is the basic building block in Verilog. A module can be an element or a collection of lower-level design blocks. A module provides the necessary functionality to the higher-level blocks through its port interface (i.e., inputs and outputs) but hides the internal implementation. This allows the designer to modify module internals without affecting the rest of the design.

In Verilog, a module is declared by the keyword **module**. A corresponding keyword **endmodule** must appear at the end of the module definition. Each module must have a *module_name*, which is the identifier for the module, and a *module_terminal_list*, which describes the input and output terminals of the module.

```
module      <module_name> (<module_terminal_list>);
. . .
<module internals>
. . .
endmodule
```



v. Ports

Ports provide the interface by which modules can communicate with its environment. For example, the *input/output* pins of an IC chip are its ports. The environment can interact with the module only through its ports. The internals of the module are not visible to the environment. This provides a very powerful flexibility to the designer. The internals of the module can be changed without affecting the environment as long as the interface is not modified. Ports are also referred to as *terminals*.



A module definition contains an optional list of ports. If the module does not exchange any signals with the environment, there are no ports in the list.

Port Declaration

All ports in the list of ports must be declared in the module. Ports can be declared as follows:

Verilog Keyword	Type of Port
input	Input port
output	Output port
Inout	Bi-directional port

Each port in the port list is defined as **input**, **output**, or **inout**, based on the direction of the port signal. Thus, for the example of the 4-bit full adder, the port declarations will be as shown below:

```
module fulladd4 (sum, c_out, a, b, c_in);

    // Begin port declarations section
    output [3:0] sum;
    output c_out;

    input [3:0] a, b;
    input c_in;
    // End port declarations section
    ...
    <module internals>
    ...
endmodule
```

Note that all port declarations are implicitly declared as **wire** in Verilog. Thus, if port is intended to be a **wire**, it is sufficient to declare it as **output**, **input**, or **inout**. Input or inout ports are normally declared as **wire**. However, if output ports hold their value, they must be declared as **reg**. For example, consider a module for a D-type flip-flop:

```
module DFF (q, d, clk, reset);
    output reg q; // Output port q holds value; therefore, it
    // is declared as reg
    input d, clk, reset;
    ...
    ...
endmodule
```



Ports of the type **input** cannot be declared as **reg**, because **reg** variables store values and **input** ports should not store values but simply reflect the changes in the external signals they are connected to.

vi. Instances

A module provides a template from which you can create actual objects. When a module is invoked, Verilog creates a unique object from the template. Each object has its own name, variables, parameters and I/O interfaces. The process of creating objects from a module template is called *instantiation*, and the objects are called *instances*. In the example below, which shows the module for a 4- bit ripple carry counter, four instances from the T-flipflop template are created. The internals of the T_FF modules are not shown.

```
// Define the top-level module called ripple carry counter.
// It instantiates 4 T-flipflops.

module ripple_carry_counter (q, clk, reset);

output [1:0] q;
input clk, reset;

// Four instances of the module T_FF are created. Each has a
// unique name. Each instance is passed a set of signals.
// Notice that each instance is a copy of the module T_FF

T_FF tff0 (.q(q[0]), .c(clk), .reset(reset));
T_FF tff1 (.q(q[1]), .c(q[0]), .reset(reset));
T_FF tff2 (.q(q[2]), .c(q[1]), .reset(reset));
T_FF tff3 (.q(q[3]), .c(q[2]), .reset(reset));

endmodule
```



The port name of a calling module (the module being instantiated) comes first. In above example, the port **c** of a T_FF tff0 module is connected to the port **clk** of a ripple_carry_counter.



For further details about Verilog HDL, go through the references provided in syllabus.

vii. Assignment Operator

A continuous assignment is the most basic statement in dataflow modeling, used to drive a value onto a net. This assignment replaces gates in the description of the circuit and describes the circuit at a higher level of abstraction. The assignment statement starts with the keyword **assign**. For example,

```
assign out = i1 & i2;
```



The left-hand side of an assignment must always be a scalar or vector net or a concatenation of scalar and vector nets. It cannot be a scalar or vector register.

The values of signals `i1` and `i2` are ANDed and assigned to the output `out`. Continuous assignments are always active. The assignment expression is evaluated as soon as one of the right-hand-side operands changes and the value is assigned to the left-hand-side net. The operands on the right-hand side can be registers or nets or function calls. Registers or nets can be scalars or vectors.

Note: Before moving to part b, attempt Quiz on LMS in Lab 5 module.

b. Hands-on Verilog HDL

We will develop an example AND gate using the concepts learnt in previous section. Diagram of a two input AND gate is given in Figure 5.1.



Figure 5.1 - Top-level diagram of an AND gate.



Following instructions are demonstrated in this [Video tutorial](#). You can watch the video or follow the instructions below for part b.

To create our first programming file, we will be using Vivado. A welcome screen will open as shown in Figure 5.2. Now we will create our first project by clicking on new project

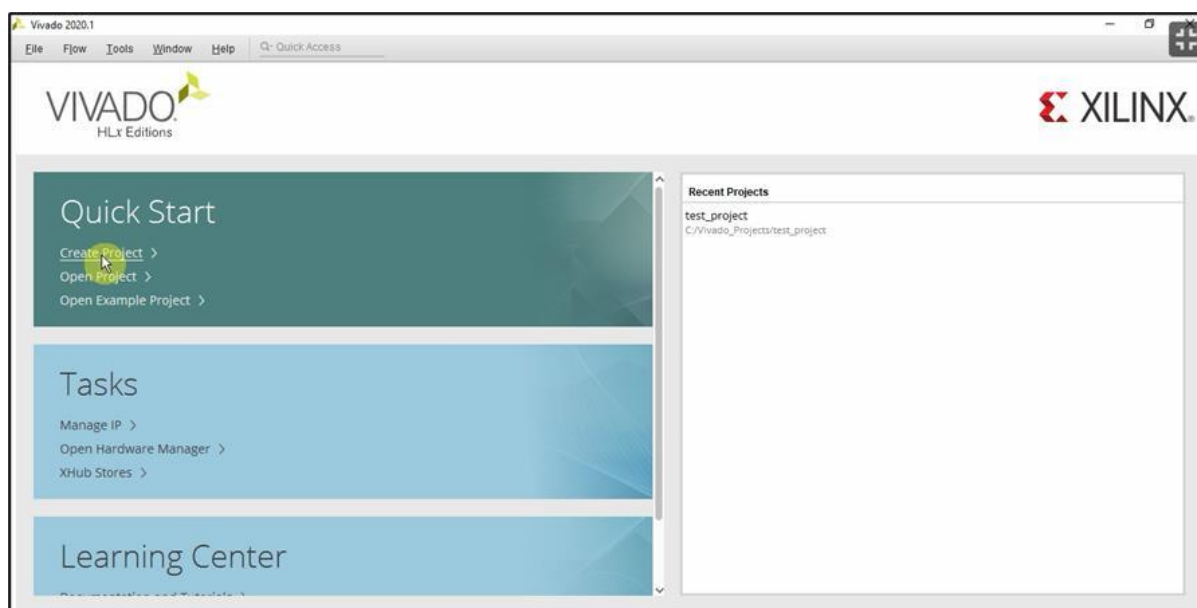


Figure 5.2 Xilinx Vivado Welcome Screen



Now name your first project. You are free to choose any name for your project. However, for uniformity basis, we will name our first file as *test_and_gate*. Also determine the destination, where you want to save your project in project location field. Once you have finalized the name and destination. Click Next

From the next menu we will choose RTL Project option, and will click next

Now we need to add sources. For this purpose, click on create file Choose Verilog as language type and create a file *myAND* as show in Figure 5.3. Click OK and Next

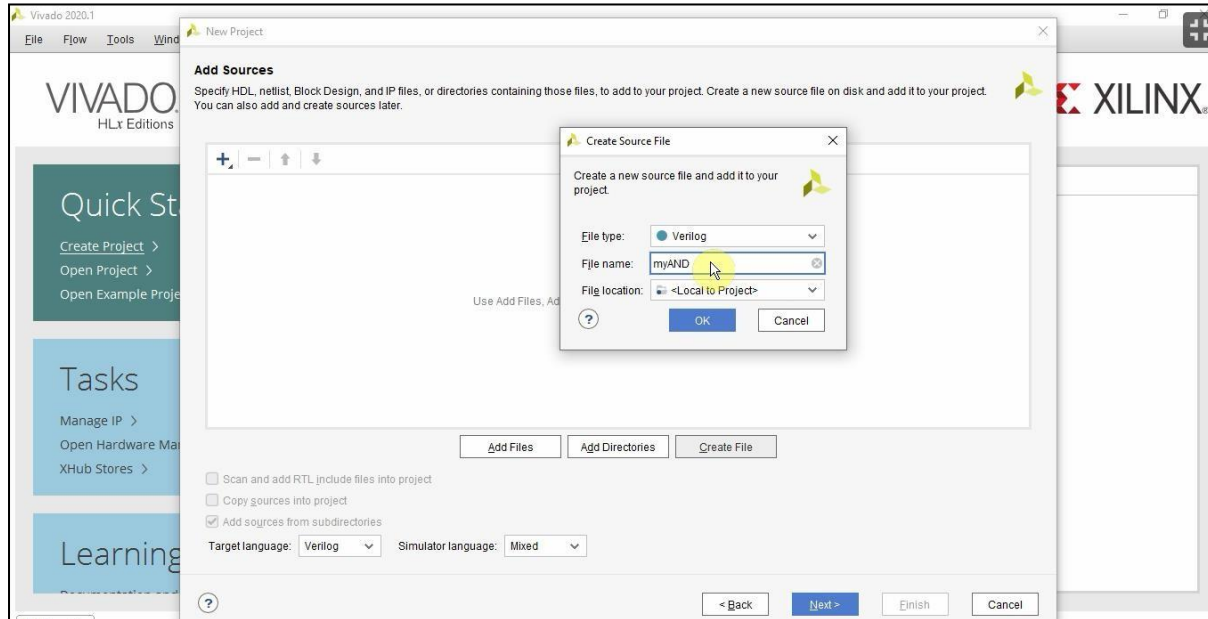


Figure 5.3 Adding Sources

Now in next step we will choose our board and constraints. As we will be using **c** board in this lab, which belongs to **General Purpose Artix-7 Family**. From the package select **cpg236**. For package part select **xc7a35tcpg236-1** as shown in Figure 5.4. and click next and press finish. The software will take some time to initialize your project

Now we need to define output and inputs for our program. As we know we will have two inputs and one output for the AND gate. Let's name our inputs as *Sw1* and *Sw2* and output as *LED* as shown in Figure 5.5

Click OK and a new screen will appear showing different parameters that have been selected by us in previous step. Under design sources, we will find our programming file as *myAND.V*. Double click the file, and it will open an editor for us to code our first program

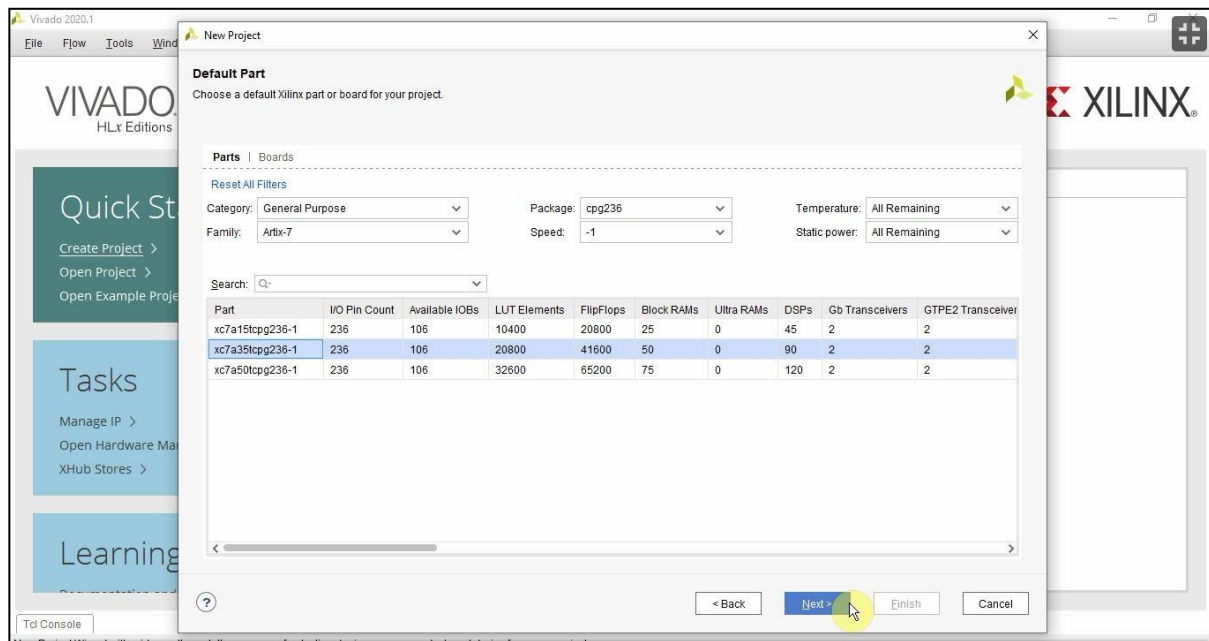


Figure 5.4 Selecting Default Parts

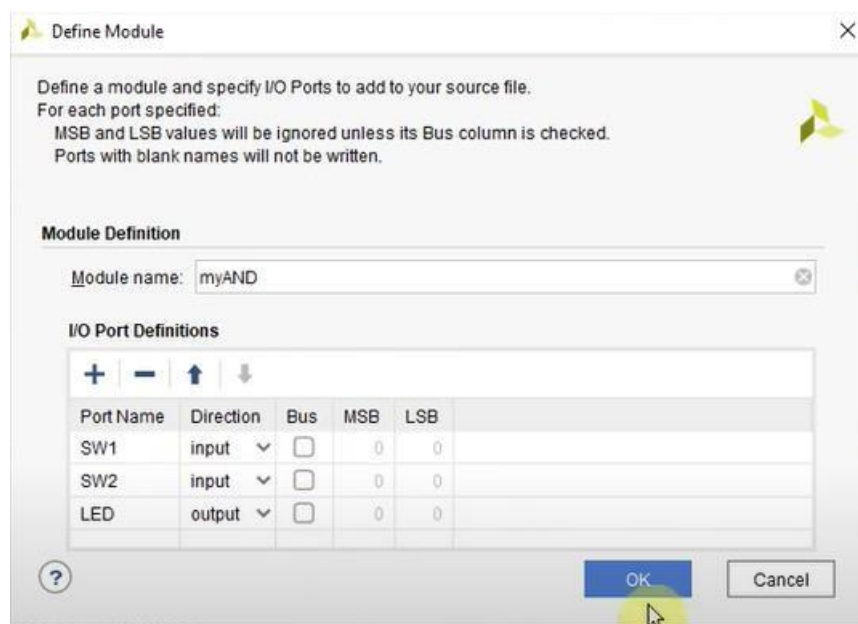


Figure 5.5 Defining input and output Ports

i. AND gate in Verilog

We will write the Verilog description of a two input AND gate. The Verilog code of AND gate is shown below

```
module myAND(
    input SW1,
```



```

    input SW2,
    output LED

);
and u1 (LED, SW1, SW2);
endmodule

```

The design of two input AND gate module is complete.



If you need to see what is implemented inside module, click on “Open Elaborate Design” in “RTL Analysis”. If there is no error, the software will provide the schematic of our AND gate circuit.

c. Functional Verification

Once a design block is completed, it must be tested. The functionality of a design block can be tested by applying stimulus and checking results. We call such a block the *stimulus* block or more commonly a *test bench*. It is good practice to keep the stimulus and design block separate. The stimulus block will also be written in Verilog. Different test benches can be used to thoroughly test the design block.

i. Creating Testbench/Stimulus

Two styles of stimulus application are possible. In the first style, the stimulus block instantiates the design block and directly drives the signals in the design block. In Figure 5.6, the stimulus block becomes the top-level block. It manipulates the signals A and B, and checks and displays output signal Y.

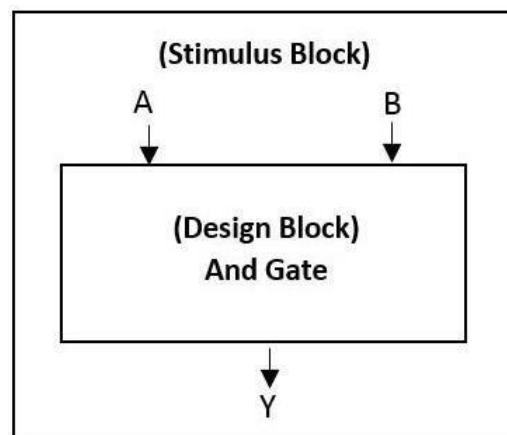


Figure 5.6 - Stimulus block instantiates design block.

The second style of applying stimulus is to instantiate both the stimulus and design blocks in a top-level dummy module. The stimulus block interacts with the design block only through the interface. This style of applying stimulus is shown in Figure 5.7. The stimulus module drives the signal `dA` and `dB` in the design block. It also checks and displays signal `dY` which is connected to the signal `Y` in the design block. The function of top-level block is to instantiate the design and stimulus blocks.

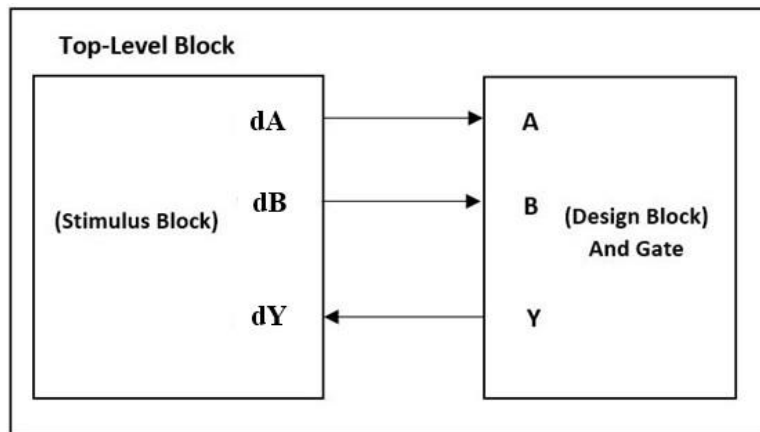


Figure 5.7 - Stimulus and Design Blocks Instantiated in a Dummy Top-Level Module.

Either stimulus style can be used effectively. In this lab, we will use the stimulus block approach shown in Figure 5.6.



In stimulus block approach (Figure 5.6), stimulus block acts like an environment to provide inputs to the design block and observe its outputs. Therefore, the stimulus block does not require any I/O ports interface.

We must now write the description of a stimulus block to check if the design module `and_gate` is functioning correctly.

The Verilog module of the testbench for testing `and_gate` design is shown below. Notice that the `testbench_AND()` module doesn't have any I/O ports. We only have internal connections declared as a wire/reg. Now the Design Under Verification (DUV) is instantiated and its ports are connected with the stimulus signals which, in our case, are `dA`, `dB` and `dY`.



To create test bench on Xilinx Vivado, follow the instructions given in this [Video tutorial](#).

In order to add test bench open the simulation sources under the simulation tab. Now add a source to the simulation source (Figure 5.8). This time select “add or create simulation sources”. Now name your file as `testbench_AND` and add testbench code.

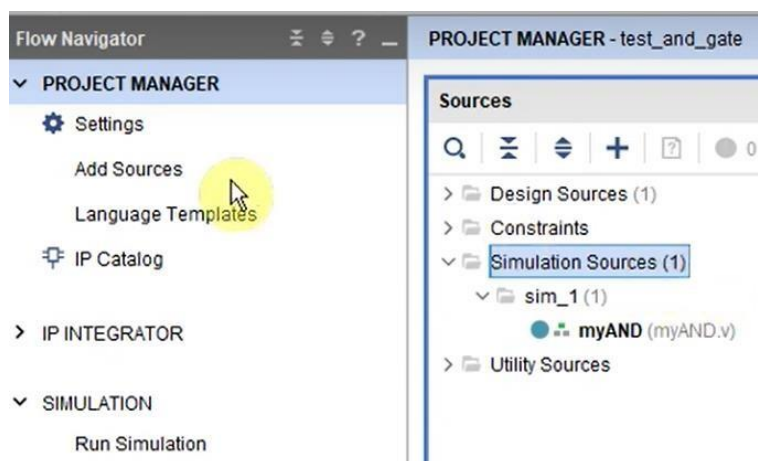


Figure 5.8 – Add Source for Simulation Sources.



Given below is the code for test bench. There are few syntax errors in the code below, correct the errors and write your testbench.

```
`timescale 1ns / 1ps module
testbench_AND();

reg [1:0] mySw;
wire myLED;

myAND module_u_test (mySw[1],mySW[0],myLED);

initial

begin

#100 mySW = 2'b00;
#100 mySW = 2'b01;
#100 mySW = 2'b10;
#100 mySW = 2'b11;
end
endmodule
```



#100 defines change of state of reg variable after the 100 ns of last instruction.

Once you have written your code, run the simulation to view timing diagram. Attach snapshot of timing diagram with your lab submission.

There is another way of writing testbench, this method is easier and there is a less chance of errors when writing testbench this way, write both the codes for testbench and state your observations (a sentence) in your report:

```
`timescale 1ns / 1ps
module
testbench_AND();

reg A,B;
wire myLED;

myAND module_u_test (A,B,myLED);

initial
begin
#100 A = 1'b0;
      B = 1'b0;

#100 A = 1'b0;
      B = 1'b1;

#100 A = 1'b1;
      B = 1'b0;
```



```
#100 A = 1'b1;
      B = 1'b1
end
endmodule
```

Exercise

For each task stated below, provide

- Verilog codes for design and test bench (Either attach the snippet of complete code or copy the code in document, make sure the word process doesn't alter the symbols or characters.)
- Snippet of timing diagram and log window depicting each combination of inputs.

Bitwise operators are given in Table 5.1

1. Create a module for OR gate and name it `or_gate`, create a test bench and verify its functionality. a
2. Create a module for XOR gate (name it `xor_gate`) and instantiate the `and_gate` and `or_gate` modules in it. Use following expression to implement XOR gate.

$$F = \bar{A}B + A\bar{B}$$

Create a test bench and verify the functionality.

3. Update the design file to implement full adder using designed basic gates. Create a test bench and verify the functionality.

Table 5.1

Types of Bitwise Operators	
Operator	Name
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left-Shift
>>	Right-Shift

Assessment Rubrics

Marks Distribution:

		LR2 Code	LR5 Results	LR7 Viva	LR9 Report
In-lab	Quiz	----	----	10 points	----
	Task	10 points	10 points	----	10 Points
Exercise	Task 1	10 points	10 points	----	
	Task 2	10 points	10 points	----	
	Task 3	10 points	10 points	----	
Total = 100		40	40	10	100 Points

Marks Obtained:

		LR2 Code	LR5 Results	LR7 Viva	LR9 Report
In-lab	Quiz	----	----		----
	Task			----	
Exercise	Task 1			----	
	Task 2			----	
	Task 3			----	
Total =					

Lab Evaluation Rubrics

#	Assessment Elements	Level 1: Unsatisfactory	Level 2: Developing	Level 3: Good	Level 4: Exemplary
LR2	Program/Code/ Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
LR7	Viva	Response shows a complete lack of understanding of the assigned / completed task	Response shows shallow understanding of the assigned task	Response shows substantial understanding of the assigned task	Response shows complete understanding of the completed task. The student is able to explain all the related concepts.
LR9	Report	No summary provided. The number/amount of tasks completed below the level of satisfaction and/or submitted late	Couldn't provide good summary of in-lab tasks. Some major tasks were completed but not explained well. Submission on time. Some major plots and figures provided	Good summary of In-lab tasks. All major tasks completed except few minor ones. The work is supported by some decent explanations, Submission on time, All necessary plots, and figures provided	Outstanding Summary of In-Lab tasks. All task completed and explained well, submitted on time, good presentation of plots and figure with proper label, titles and captions